

**Memoria de la práctica sobre
Heat Equation con el algoritmo Jacobi**



TGA FIB UPC 2025-2026
David Castro y David Víctor

1. Introducción

1.1 Heat Equation

La ecuación del calor constituye uno de los pilares fundamentales de la física matemática y la termodinámica desde su formulación original por Joseph Fourier en 1822. Esta ecuación en derivadas parciales de tipo parabólico describe con precisión cómo la temperatura se distribuye y evoluciona en una región determinada a lo largo del tiempo. Su desarrollo no solo permitió modelar el comportamiento del flujo térmico, sino que también sentó las bases para el análisis de Fourier, transformando nuestra comprensión de la difusión en medios continuos. Físicamente, la ecuación establece que la tasa de cambio de la temperatura en un punto es proporcional a la curvatura de su perfil de temperatura, lo que implica que el calor fluye de manera natural desde las zonas de mayor energía hacia las de menor energía, buscando siempre un estado de equilibrio térmico.

1.2 Algoritmo Jacobi

Dado que una computadora no puede calcular infinitos puntos en un medio continuo, dividimos el dominio (por ejemplo, una placa metálica) en una rejilla o malla de puntos con distancias uniformes. En lugar de tratar con derivadas, utilizamos el Método de Diferencias Finitas, donde aproximamos los cambios de temperatura comparando cada punto con sus vecinos inmediatos. En un estado de equilibrio (donde la temperatura ya no cambia en el tiempo), la ecuación del calor se reduce a la ecuación de Laplace. Para un punto en una rejilla bidimensional, la aproximación discreta establece que su temperatura debe ser, idealmente, el promedio de sus cuatro vecinos.

El **Método de Jacobi** es uno de los algoritmos iterativos más fundamentales para resolver este sistema de ecuaciones. Su lógica es sencilla y altamente intuitiva para la programación:

1. **Inicialización:** Se asigna un valor inicial a todos los puntos de la malla (por ejemplo, 0°C) y se fijan los valores constantes en los bordes (condiciones de contorno).
2. **Iteración:** El algoritmo recorre cada punto de la malla y calcula una "nueva" temperatura basada en los valores "viejos" de sus vecinos.
3. **Actualización simultánea:** Una característica crítica de Jacobi es que **no mezcla** valores viejos con nuevos durante el mismo paso. Se necesita una copia de la malla para leer los datos y otra para escribir los resultados de la iteración actual.
4. **Convergencia:** El proceso se repite miles de veces hasta que la diferencia entre la temperatura nueva y la anterior es tan pequeña que consideramos que el sistema ha alcanzado el equilibrio.

La regla de actualización en cada paso para cada punto (i,j) se define como:

$$u_{\text{new}}[i,j] = (u[i+1,j] + u[i-1,j] + u[i,j+1] + u[i,j-1]) / 4$$

El algoritmo de Jacobi es apreciado en computación por su **paralelismo natural**, ya que el cálculo de un punto no depende de los resultados de sus vecinos en esa misma iteración, lo que permite usar tarjetas gráficas (GPU) para acelerar la simulación. Sin embargo, es computacionalmente costoso y más lento en converger que variantes más modernas como el método de Gauss-Seidel o el de Sobre-relajación Sucesiva (SOR).

2. Implementación Cuda en 1 GPU

Partiendo del código secuencial de referencia, migramos la carga computacional intensiva a la GPU, manteniendo el control de flujo en el Host. El diseño prioriza la permanencia de las matrices principales (u y uhelp) en memoria global durante todo el proceso iterativo. Esto minimiza las transferencias Host-Device estrictamente a la inicialización y recuperación final, maximizando el ancho de banda efectivo.

2.1 Versiones

Implementamos inicialmente versiones por filas y columnas donde cada hilo procesa vectores completos iterativamente. Sin embargo, debido al uso ineficiente del ancho de banda, evolucionamos hacia una descomposición por bloques donde cada hilo calcula un único punto. Esta estrategia maximiza el paralelismo al distribuir la carga entre más hilos con menor carga individual.

2.2. Implementación por Filas y Columnas

Flujo de ejecución y cálculo del residuo

Nuestra primera aproximación a la paralelización ha consistido en asignar una fila o columna a cada thread.

En la versión por filas (jacobi_row_kernel), cada thread es responsable de procesar una fila completa de la matriz, iterando elemento a elemento de izquierda a derecha, la lógica es análoga para la versión por columnas.

Conceptualmente, la función solve_cuda orquesta este proceso en dos etapas por iteración. Primero, se lanza el kernel de Jacobi, donde cada hilo calcula los nuevos valores de temperatura (u_new) y, simultáneamente, acumula en un registro local la suma de las diferencias al cuadrado de su fila o columna. Al finalizar del bucle, cada hilo escribe este resultado parcial en un vector de memoria global (diffs_dev). Posteriormente, lanzamos el kernel de reducción (reduction_kernel07), inspirado en la versión que hemos usado en las sesiones de laboratorio, que suma este vector de resultados parciales en un único valor (residual) que copiamos al Host para verificar la convergencia.

Necesitamos que el valor residual sea más pequeño que un valor límite que fijamos nosotros para que el algoritmo deje de iterar, también podemos fijar el número máximo de iteraciones que queremos que haga como mucho.

Optimización con Shared Memory (Cooperación entre hilos)

Para mitigar la latencia de la Memoria Global, implementamos cooperación en una dimensión. Mediante un buffer en memoria compartida (`s_col`), los hilos cargan cooperativamente la columna actual de datos. Tras sincronizar (`__syncthreads()`), los vecinos superior e inferior se leen directamente de esta memoria rápida, ahorrando dos accesos a memoria global por cada elemento procesado.

Veremos si esta optimización tiene un efecto positivo en los tiempos de ejecución.

Gestión eficiente con Pinned Memory

Para maximizar el ancho de banda en las transferencias, sustituimos la asignación estándar por `cudaMallocHost` (bajo flag `PINNED`). Esto aloja las matrices en memoria paginada del sistema, habilitando el acceso vía DMA (Direct Memory Access) sin intervención de la CPU. Esto acelera significativamente tanto la carga inicial de datos como la recuperación del resultado final.

Paralelismo asíncrono mediante Streams

Finalmente, adaptamos estos kernels para soportar ejecución asíncrona mediante CUDA Streams. Modificamos los kernels para aceptar un parámetro de desplazamiento (`offset`), lo que nos permite dividir la matriz en "franjas" horizontales o verticales. En el bucle de control, lanzamos múltiples instancias del kernel, cada una asociada a un stream diferente y encargada de una porción distinta de la matriz. Aunque la dependencia de datos intrínseca del método de Jacobi (necesitamos la matriz completa del paso t para calcular $t+1$) impide solapar iteraciones temporales, esta técnica nos permite solapar la ejecución de los kernels con operaciones de la CPU o transferencias si las hubiera, además de maximizar la ocupación de la GPU al tener múltiples colas de trabajo activas simultáneamente.

Todas estas operaciones versiones de fila/columnas se pueden probar modificando los flags que hemos puesto en [misc.cu](#) , [heat.cu](#) y [solver.cu](#) .

2.3. Implementación por Bloques

Descomposición espacial 2D (Tiling)

Tras analizar las limitaciones de las versiones por filas y columnas, dimos el paso lógico hacia una descomposición bidimensional, conocida como tiling. En esta versión (`jacobi_shared_block_kernel`), dividimos la matriz en bloques cuadrados de 16x16 hilos. Esta estructura se ajusta de manera natural a la física del problema, ya que el calor se propaga en todas las direcciones del plano, no solo a lo largo de filas o columnas.

Al asignar a cada hilo el cálculo de un único punto de la malla, logramos un equilibrio mucho mejor entre la carga computacional y los accesos a memoria, permitiendo escalar el problema a resoluciones mucho mayores sin saturar los recursos de la GPU. Cada bloque de threads se encarga de una parcela.

Gestión avanzada de Shared Memory con Halos

Para eliminar la latencia de memoria global, cada bloque carga cooperativamente un tile extendido de 18×18 en memoria compartida (16×16 datos propios + halo exterior). Cada hilo carga su celda y, si aplica, el vecino del borde. Tras sincronizar (`__syncthreads()`), garantizamos que todo el bloque necesario reside en memoria rápida, permitiendo que el cálculo del stencil se ejecute sin acceder a memoria global.

Reducción jerárquica intra-bloque

Para evitar saturar la memoria global con escrituras individuales, implementamos una reducción parcial en memoria compartida. Los hilos suman cooperativamente sus errores locales mediante un algoritmo de árbol, de modo que solo el hilo 0 escribe el total acumulado del bloque (16×16). Esto reduce el número de escrituras en memoria global.

Adaptación de Streams a bloques 2D

Para integrar esta versión con la ejecución asíncrona mediante Streams, adaptamos el sistema de offsets. Dado que ahora trabajamos con una rejilla 2D de bloques, el offset ya no representa una fila simple, sino un desplazamiento en la coordenada Y de los bloques (`by_offset`). Cuando usamos múltiples streams, dividimos la rejilla de bloques en franjas horizontales y cada stream lanza un kernel que procesa un subconjunto de estas filas de bloques. Esto nos permite mantener la misma lógica de solapamiento de cómputo y transferencias que diseñamos para las versiones anteriores, pero aplicada a esta versión más eficiente basada en bloques.

2.4 Optimizaciones:

1. Coalescencia de Memoria (Memory Coalescing)

El rendimiento depende críticamente del patrón de acceso. En la versión Row Kernel, el acceso es disperso porque hilos consecutivos leen filas distintas, impidiendo agrupar transacciones. En cambio, en Column y Block Kernel, aseguramos que hilos adyacentes accedan a direcciones contiguas ($j, j+1$). Esto garantiza acceso coalescente, permitiendo que la GPU sirva las peticiones de todo un warp (32 hilos) en una única transacción y maximizando el ancho de banda.

2. Optimización de Memoria: Intercambio de punteros

Para evitar la costosa copia de datos entre matrices ($O(N^2)$) necesaria al finalizar cada iteración, simplemente intercambiamos los punteros de lectura (`u_dev`) y escritura (`uhelp_dev`) en el Host. Esto elimina por completo la transferencia de datos, transformando la actualización del estado en una operación inmediata de coste $O(1)$.

3. Reducción Global de las Diferencias

1. Para sumar eficientemente los residuos parciales, implementamos una reducción en cascada utilizando el `reduction_kernel07`. El proceso consta de una primera pasada masiva que reduce el dominio a un vector de sumas parciales (ajustando dinámicamente el número de bloques para evitar hilos ociosos si N es pequeño), seguida de una segunda pasada con un único bloque que colapsa estos resultados en el valor final. Esto minimiza la latencia y la sincronización global a solo dos llamadas al kernel.

3. Implementación Cuda en varias GPU

Tras analizar el rendimiento de las distintas configuraciones en una única GPU, hemos determinado que la implementación más eficiente es la división de trabajo por bloques mediante el uso de memoria compartida. Basándonos en estos resultados, en este apartado escalaremos dicha versión óptima para evaluar su comportamiento en sistemas de 1, 2 y 4 GPUs, prescindiendo en esta fase del uso de streams y usando memoria Pinned.

El objetivo principal es reducir el tiempo de ejecución global permitiendo que cada GPU procese una porción de la malla de forma simultánea.

3.1 Estrategia de División

Para esta implementación multi-GPU, hemos optado por una división por franjas horizontales. Si disponemos de N GPUs, dividimos la matriz de dimensiones 2048×2048 (por ejemplo) en N bloques de filas de tamaño aproximado $(2048/N) \times 2048$.

Cada GPU es responsable de calcular los nuevos valores de temperatura para su franja asignada. Al igual que en la versión de una sola GPU, el control del flujo iterativo y la verificación de la convergencia siguen siendo tarea del Host.

3.2 Trabajo Multi-GPU

El flujo de ejecución de las GPUs es el siguiente:

1. **Selección de Dispositivo:** Mediante `cudaSetDevice(i)`, el programa activa el contexto de la GPU correspondiente.
2. **Lanzamiento de Kernels en Paralelo:** Se lanzan los kernels de Jacobi de manera que cada GPU trabaje sobre su subdominio.
3. **Reducción del Residuo:** Cada dispositivo calcula su error local sobre su franja de filas. Posteriormente, estos resultados parciales se combinan para obtener el residuo global y determinar si el sistema ha alcanzado el equilibrio térmico.

3.3 El Reto de las Fronteras

Para resolver las dependencias de datos en los bordes de cada dominio, implementamos acceso **Peer-to-Peer (P2P)**. Mediante `cudaDeviceEnablePeerAccess`, realizamos el intercambio de filas a través de copias directas entre GPUs. Esto elimina la necesidad de triangular los datos por la memoria del Host (CPU), evitando cuellos de botella en el bus y reduciendo drásticamente la latencia de sincronización.

4. Resultados esperados

4.1 Resultados en implementaciones de 1 GPU

Se evaluó el tiempo de ejecución de la resolución de la ecuación de calor, cronometrando desde el inicio del bucle en la CPU hasta la convergencia del sistema o el cumplimiento del límite de 250,000 iteraciones.

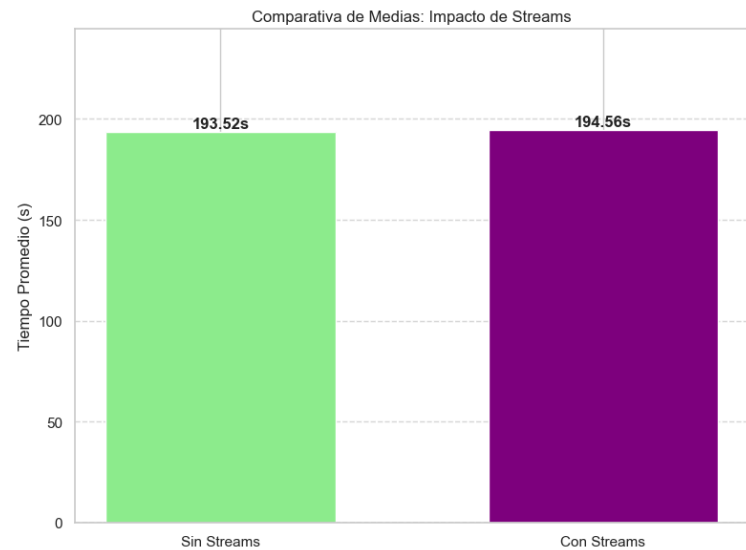
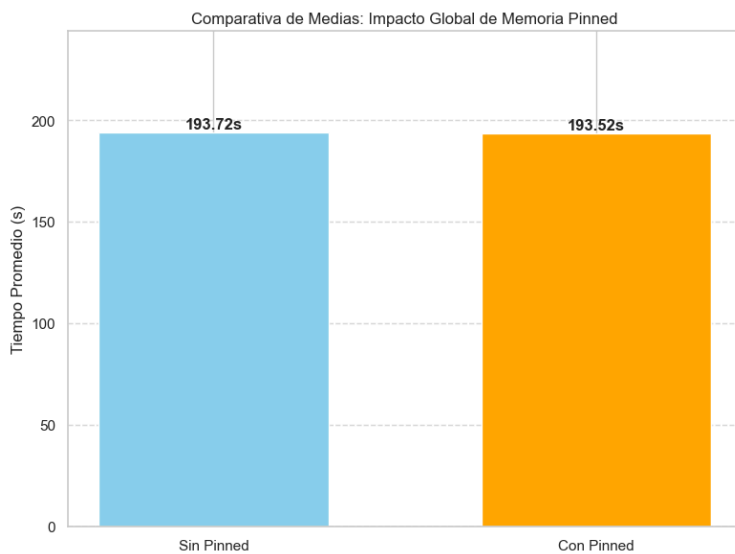
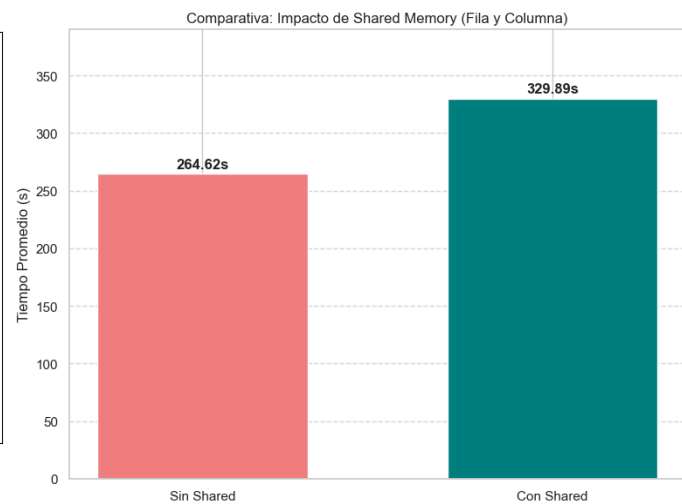
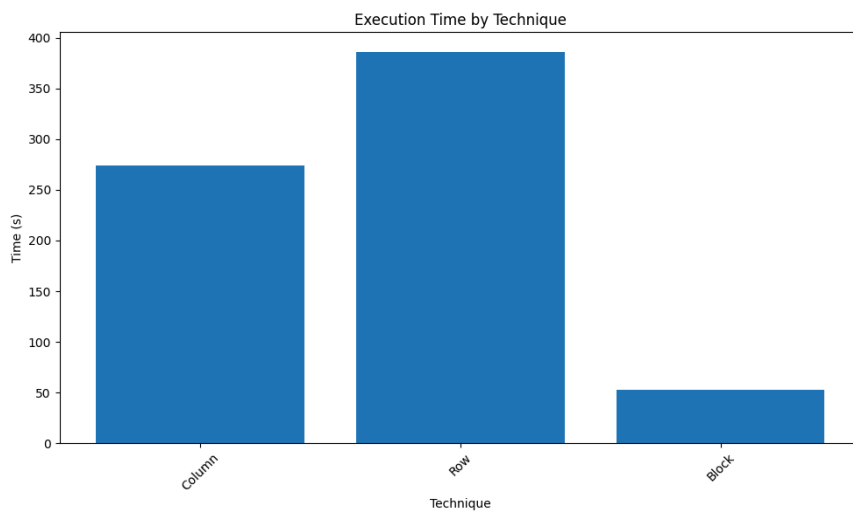
Para las condiciones iniciales, se utilizaron los dos focos de calor proporcionados en el archivo `test.dat`, incrementando su área a 2.0 unidades. Asimismo, se emplearon mallas de 2048 x 2048, un tamaño superior al original, con el fin de analizar el impacto de la carga computacional en el rendimiento. Todas las pruebas se realizaron utilizando una GPU NVIDIA RTX 3080. El block size definido fue de 256 threads.

Estos fueron los resultados observados con la configuración comentada:

Técnicas	Work	Pinned	Shared	Streams	T
1	Column	No	No	No	186.86s
2	Column	Yes	No	No	187.01s
3	Column	Yes	Yes	No	273.71s
4	Column	Yes	No	Yes	187.21s
5	Row	No	No	No	342.98s
6	Row	Yes	No	No	342.24s

7	Row	Yes	Yes	No	386.07s
8	Row	Yes	No	Yes	343.77s
9	Block	No	Yes	No	51.33s
10	Block	Yes	Yes	No	51.31s
11	Block	Yes	Yes	Yes	52.69s

Hemos graficado tanto la comparación entre trabajo de thread y las demás técnicas:



La comparativa entre las técnicas fundamentales revela que el trabajo asignado a cada thread y la disposición de los datos en memoria son los factores determinantes en el tiempo de ejecución.

Técnica de Fila (Row) vs. Columna (Column)

Se observa una disparidad significativa en los tiempos de ejecución entre estas dos técnicas, siendo la técnica de Row (~342s) considerablemente más lenta que la de Column (~187s).

- **Coalescencia de Memoria:** La superioridad de la técnica Column se debe a la coalescencia. En esta configuración, los hilos de un mismo *warp* acceden a direcciones de memoria contiguas. Esto permite que el hardware de la GPU combine múltiples solicitudes de memoria en una sola transacción, optimizando el ancho de banda.
- **Acceso no coalescente en Row:** En la técnica de Row, el patrón de acceso es disperso o "a saltos" (*strided*). Cada hilo solicita datos que se encuentran alejados entre sí en la memoria global, lo que obliga a la GPU a realizar múltiples transacciones para una sola instrucción, degradando drásticamente el rendimiento.

Técnica por Bloques (Block)

La técnica de Block presenta el mejor rendimiento global con tiempos consistentes cercanos a los 51s.

- **Eficiencia de Acceso:** Esta técnica maximiza la localidad de los datos. Al procesar sub-matrices, se asegura que los accesos sean secuenciales y seguidos, permitiendo un uso extremadamente eficiente de la jerarquía de caché de la GPU y reduciendo al mínimo la latencia de la memoria global. Asimismo, la estructuración del trabajo en submatrices de 16x16 favorece la escalabilidad del algoritmo. Esta configuración permite ajustar dinámicamente el número de hilos y bloques en función de las capacidades del hardware, optimizando la ocupación de la GPU y maximizando el rendimiento global del kernel.

Memoria Pinned

El uso de Pinned Memory muestra un impacto marginal en los tiempos totales. En los casos base, la variación es inferior al 0.1%. El impacto es nulo y se debe a que sólo contemplamos el tiempo de ejecución del cálculo de la solución. Las transferencias no se tienen en cuenta y la variación es debido a aleatoriedad.

Memoria Shared

Los resultados muestran un comportamiento contraproducente en las técnicas de Row y Column al activar Shared Memory:

- En Column, el tiempo se incrementa de 187.01s a 273.71s.
- En Row, el tiempo sube de 342.24s a 386.07s.

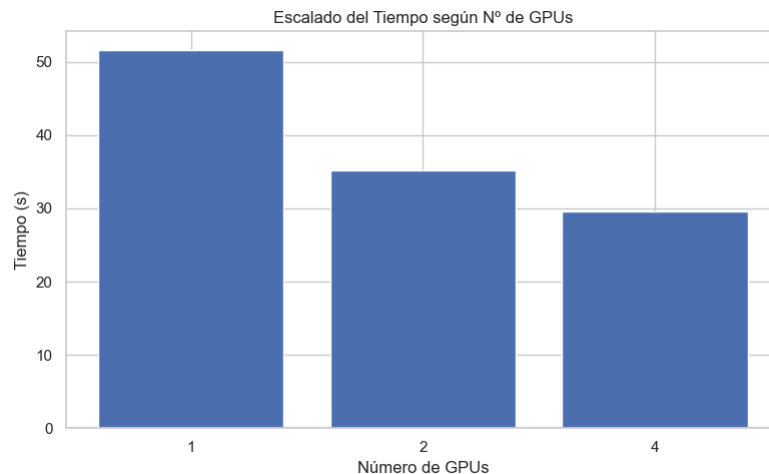
Este incremento sugiere que el coste de transferencia de los datos desde la memoria global a la memoria compartida no se ve compensado. El proceso de cargar una fila o columna completa en la memoria compartida introduce una latencia superior al ahorro obtenido, dado que solo se optimizan tres accesos a la memoria global, lo que no justifica el overhead de la operación. Otra implementación igual sí conseguiría ser beneficiosa.

Ejecución con streams

La implementación de Streams no ha reportado mejoras de rendimiento en el escenario actual, manteniendo tiempos equivalentes o ligeramente superiores a sus versiones secuenciales. Este comportamiento se debe a que la segmentación en streams se limitó exclusivamente al lanzamiento de los kernels.

4.2 Resultados en implementaciones multi-GPU

Nº GPU	T
1	51.57s
2	35.20s
4	29.61s



Tras identificar la técnica de Block como la más eficiente en términos de acceso a memoria y rendimiento, se procedió a evaluar su capacidad de escalabilidad mediante la distribución de carga en múltiples dispositivos (Multi-GPU). Los resultados muestran una reducción del tiempo de ejecución, aunque con un comportamiento de rendimiento decreciente.

A partir de los tiempos registrados, se observa el siguiente factor de aceleración respecto a la ejecución en una sola unidad:

- Configuración 2 GPUs: Se logra una reducción de 51.57s a 35.20s, lo que representa un *speedup* de 1.47 y una eficiencia de escalado del 73.5%.
- Configuración 4 GPUs: El tiempo desciende a 29.61s, alcanzando un *speedup* acumulado de 1.74 respecto a una GPU. Sin embargo, la eficiencia de escalado cae significativamente a un 43.5%.

Para el tamaño del problema actual, el sistema muestra signos de saturación al alcanzar las 4 GPUs.